

プログラミング入門2

第2回 複合文、繰り返し

篠埜 功

今回の講義内容

- 複合文 --- 複数の文を1つの文にまとめる。
 - 文を書くところに2つ以上の文を書くときに使う。
 - (例) if文の分岐先の文のところに2つ以上の文を書く場合など。
- 繰り返し
 - 何らかの処理を何度も繰り返し行う --- 繰り返しは命令型言語における基本的な機構。

複合文(ブロック)(教科書 p. 60)

複合文の構文

{ 0個以上の宣言あるいは文の並び }

複合文の意味

複合文の中の文を順番に実行

重要

宣言された変数 x の有効範囲は、 x の宣言の場所から複合文の最後まで。(ただし、複合文中に複合文があってそこで同じ名前の変数が宣言された場合はそこは除く。)(教科書 p. 155, p. 173)

複合文の例

- `{ }`
--- 宣言も文もない複合文
- `{ printf ("test¥n"); }`
--- 文が1つの複合文
- `{ int x; x = 5; }`
--- 宣言1つ、文1つの複合文
- `{ int x; x = 5; printf ("%d", x); }`
--- 宣言1つ、文2つの複合文
- `{ int x; int y; x=5; y=3; printf ("%d", x); }`
--- 宣言2つ、文3つの複合文

複合文を使ったプログラム例(1)

- これまでのプログラムはすべて複合文を使っていた。

```
#include <stdio.h>
int main (void)
{
    printf ("%d¥n", 15);
    return 0;
}
```

赤字の部分は宣言無し、文2つの複合文である。
これは、main関数の本体を成している。(関数の説明の回でもう一度説明する)

複合文を使ったプログラム例(2)

```
#include <stdio.h>
int main (void)
{
    int x;
    x=3;
    printf ("x=%d¥n", x);
    return 0;
}
```

赤字の部分は宣言1つ、文3つの複合文である。
これは、main関数の本体を成している。

注意(重要)

変数宣言をしただけだと、その変数の値は不定(ごみが入っている)。
変数に値を格納する前に変数の値を読み取ってはいけない。(そういうプログラムを書いたら、動作が**未定義(undefined)**となる。)(教科書 p. 14)

```
#include <stdio.h>

int main (void)
{
    int x;

    printf ("x=%d¥n", x);  /* xの値は不定(ごみが入っている)。この行が実行されると変な値が画面に出てくることがほとんどだが、エラーになって実行が止まったり、あるいはコンパイル時にエラーになるかもしれない。何が起こるかはコンパイラによって異なる。(つまり、コンパイラの実装者が好きにしていよう。) */

    return 0;
}
```

複合文を使ったプログラム例(3)

```
#include <stdio.h>
int main (void)
{
    int x;
    x=3;
    {
        int y;
        y=5;
        printf ("x=%d¥n", x);
        printf ("y=%d¥n", y);
    }
    return 0;
}
```

赤字の部分は宣言1つ、文3つの複合文であり、外側の複合文の2つ目の文を成している。変数yの有効範囲は赤色の部分のみである。有効範囲を限ると、プログラムの可読性が上がる、同じ名前の変数を別の変数として用いることができる等のメリットがある(この例ではメリットは感じられないが)。

複合文を使ったプログラム例(4)

```
#include <stdio.h>

int main (void)
{
    int x;
    x=3;
    {
        int x;
        x=5;
        printf ("x=%d¥n", x);  /* xの値は5 */
    }
    printf ("x=%d¥n", x);      /* xの値は3 */
    return 0;
}
```

内側の複合文(赤色の部分)で、外側で既に宣言されている変数xと同じ名前の変数を宣言している。名前は同じでも、別の変数であることに注意。

この例では、内側の複合文においては、外側の変数xは見えなくなる。(教科書 p. 173)
このことを英語でshadowingという。

複合文を使ったプログラム例(5)

```
/* 整数値を入力し、正かどうか判定 */
#include <stdio.h>
int main (void)
{
    int x;
    printf ("Input an integer: ");
    scanf ("%d", &x);
    if (x>0)
        printf ("%d is greater than 0.¥n", x);
    else {
        printf ("%d is less than or equal to 0.¥n", x);
    }
    return 0;
}
```

赤字の部分は宣言0個、文1個の複合文であり、if文のelse部分を成す。

文1つからなる複合文は中括弧{}をはずしても意味は変わらない。

複合文を使ったプログラム例(6)

(打ち込んで確認)

```
/* 整数値を入力し、それが正の場合、もう一つの整数値を入力し、それらの積を表示 */
```

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int x;
```

```
    printf ("Input an integer: ");
```

```
    scanf ("%d", &x);
```

```
    if (x>0) {
```

```
        int y;
```

```
        printf ("Input an integer: ");
```

```
        scanf ("%d", &y);
```

```
        printf ("%d * %d = %d.\n", x, y, x*y);
```

```
    }
```

```
    return 0;
```

```
}
```

赤字の部分は宣言1つ、文3つの複合文であり、if文(elseパート無し)のif文の本体を成す。変数yの有効範囲が赤字の部分だけであることに注意。

補足

1990年のISO規格では、複合文の先頭部分に変数宣言を0個以上書き、そのあとで文を0個以上書くようになっていた。

複合文の構文

{ 0個以上の宣言 0個以上の文 }

複合文の意味

複合文の中の文を順番に実行

1999年のISO規格(C99と呼ばれる)以降では、複合文の中で、変数宣言は先頭部分以外に書いてもよい。

複合文を使ったプログラム例(7)

```
#include <stdio.h>
int main (void) {
    int x;
    x=3;
    int y;
    y=4;
    printf ("x=%d, y=%d\n", x, y);
    return 0;
}
```

変数yの宣言を複合文の
先頭以外で行っている。

複合文を使ったプログラム例(8)

```
#include <stdio.h>
int main (void) {
    int x;
    x=3;
    {
        printf ("x=%d¥n", x);  /* xの値は3 */
        x=100;
        int x;
        x=5;
        printf ("x=%d¥n", x);  /* xの値は5 */
    }
    printf ("x=%d¥n", x);      /* xの値は100 */
    return 0;
}
```

変数xが内側の複合文の途中で宣言されている。

繰り返し

■ 同じ処理を繰り返すには・・・

- プログラム中に同じ命令を何度も繰り返して書く？

--- 繰り返す回数が入力によって変わる場合など、対応できない。

■ 条件により、繰り返すかをどうかを決定

■ 繰り返しの構文

- while文
- for文
- do-while文

今日はwhile文を紹介する。後日for文を紹介する。do-while文については教科書を参照。

while文(教科書 p. 82)

while文の構文

while (式) 文

while文 **while (e) s** の意味

式 e を評価し、それが0の場合何もせず、0以外の場合は文 s を実行した後、while (e) s をもう一度実行する。

(つまり、式 e の値が0になるまで文 s を繰り返し実行する。)

while文を使ったプログラム例 (打ち込んで実行)

```
/* 羊を10匹まで数えたら寝る */
```

```
#include <stdio.h>
```

```
int main (void) {  
    int x;  
    x = 1;  
    while (x <= 10) {  
        printf ("羊が%d匹¥n", x);  
        x=x+1;  
    }  
    printf ("グーグー¥n");  
    return 0;  
}
```

赤字の部分は宣言0個、文2個の複合文であり、while文の本体を成している。

無限ループ(打ち込んで実行)

```
#include <stdio.h>

int main (void)
{
    int x;
    x=1;
    while (1) {
        printf ("羊が%d匹¥n", x );
        x = x + 1;
    }
    return 0;
}
```

Ctrl-c (Ctrlキーを押しながら
cを押す)で終了

ラベル、goto文（打ち込んで実行）

goto文は、プロ入2の課題ではできるだけ使わないようにしてください。ですが、知らないのはよくないので紹介します。（教科書には載っていません）

```
#include <stdio.h>
int main (void) {
    int x;
    x=1;
    aaa:
    printf ("羊が%d匹¥n", x);
    x=x+1;
    if (x<=10) goto aaa;
    printf ("ぐーぐー¥n");
    return 0;
}
```

青字の部分がラベル、赤字の部分がgoto文である。

goto文を使った無限ループ (打ち込んで実行)

```
#include <stdio.h>

int main (void)
{
    int x;
    x=1;
    aaa:
    printf ("羊が%d匹¥n", x);
    x=x+1;
    goto aaa;
    return 0;
}
```

Ctrl-c (Ctrlキーを押しながら
cを押す)で終了

連絡事項

- 履修者の皆さん: 今後のプロ入2の課題すべてについて、できる人はgoto文を使わずに書いてください。すべてのプログラムはgoto文を使わずに書くことができます。
- TAの皆さん: 今後のプロ入2の課題すべてについて、goto文を使っていても、正しければ正解にしてください。ただし、goto文を使わない書き方を教えてあげてください。

基本課題1

0以上のint型の値をキーボードから受け取り、その値を0まで1ずつ減らしながらコンマで区切って以下の例のように表示するプログラムを書け。

[実行例]

整数を入力: 5

5, 4, 3, 2, 1, 0

(注意) 0の右側にはコンマはつけないようにして下さい。

基本課題2

正の整数をキーボードから読み込み、その数の約数を小さい順にすべてコンマで区切って以下の例のように表示するプログラムを作成せよ。

[実行例]

正の整数を入力してください: 20

20の約数を小さい順に列挙すると、
1, 2, 4, 5, 10, 20 である。

(ヒント) a が b の約数かどうかは $b\%a$ の値が0かどうかで判定できる。

(注意) 最後の約数の右側にはコンマはつけないようにして下さい。

発展課題1

2つの正の整数をキーボードから受け取り、それらの最小公倍数を求め、以下のように表示するプログラムを作成せよ。

[実行例]

正の整数を入力してください: 4

正の整数を入力してください: 6

4と6の最小公倍数は12です。

発展課題1の補足

(ヒント) 2つの正の整数 a, b の最大公約数を c とすると、 $a=cd$, $b=ce$ と書ける。 a と b の最小公倍数は、 cde である。

(補足) 最大公約数の求め方については、プログラミング入門1でやったようにユークリッドの互除法を使えばよいが、単に1から順番に割っていくなど、他の方法でも構わない。

発展課題2

1つの正の整数をキーボードから受け取り、その値が素数かどうかを以下のように表示するプログラムを書け。素数とは、1と自分自身以外に正の約数を持たない、1でない正の整数のことである。

[実行例1]

正の整数を入力してください: 37

37は素数です。

[実行例2]

正の整数を入力してください: 39

39は素数ではありません。

発展課題3

銀行に1000万円を年利0.01%（複利）で預けたとする。1年ごとの残高を50年後まで以下のように表示するプログラムを書け。利息は1年に一度つくものとし、税金はかからないとする。利息は1円未満は切り捨てとする。（毎年切り捨ててください）

[実行例]

1年後: 10001000円

2年後: 10002000円

3年後: 10003000円

...

48年後: 10048092円

49年後: 10049096円

50年後: 10050100円

発展課題4

以下の例のように分数の足し算を行うプログラムを書け。すべて正の整数が入力されることを前提として良い。足し算の結果は既約分数で表示せよ。ただし、分母が1の場合も分数の形で書くものとする。

[実行例1]

分数の足し算をします。

1つ目の分数の分子: 1

1つ目の分数の分母: 2

2つ目の分数の分子: 5

2つ目の分数の分母: 6

$$1/2 + 5/6 = 4/3$$

[実行例2]

分数の足し算をします。

1つ目の分数の分子: 19

1つ目の分数の分母: 20

2つ目の分数の分子: 41

2つ目の分数の分母: 20

$$19/20 + 41/20 = 3/1$$

参考課題1（数当てゲーム）

0～9の整数をキーボードから読み込み、正解より大きいのか、小さいのか、等しいかを判定し、画面上に表示するということを正解になるまで繰り返すプログラムを作成せよ。正解は自分でプログラム記述時に決めておくものとする。

[実行例]

0～9の整数を当ててください

いくつですか? 3

正解はもっと大きいです

いくつですか? 8

正解はもっと小さいです

いくつですか? 7

正解です

（この例では正解は7としている。）

参考課題1 解答例

```
#include <stdio.h>
int main (void){
    int n;
    int ans=7;
    printf("0～9の整数を当ててください¥n");
    printf("いくつですか? ");
    scanf("%d",&n);
    while (n!=ans) {
        if(n>ans)
            printf("正解はもっと小さいです¥n");
        else if(n<ans)
            printf("正解はもっと大きいです¥n");
        printf("いくつですか? ");
        scanf("%d",&n);
    }
    printf("正解です¥n");
    return 0;
}
```

[参考]

この講義では説明ませんが、do-while文を用いると、同じ内容のprintf文("いくつですか?"の表示)とscanf文を2か所に書かなくてよくなります。

参考課題2

1以上の整数をキーボードから受け取り、その値の階乗を表示するプログラムを書け。

[実行例]

1以上の整数を入力: 6

6の階乗は720です

参考課題2解答例

```
#include<stdio.h>
int main(void){
    int n;
    int factorial;
    int i;
    factorial = 1;
    printf("1以上の整数を入力: ");
    scanf("%d", &n);
    i=n;
    while(i!=0){
        factorial = factorial * i;
        i=i-1;
    }
    printf("%dの階乗は%dです\n", n, factorial);
    return 0;
}
```


参考課題3

キーボードから整数を受け取り、0が入力されるまで入力された値を加えていき、結果を表示するプログラムを書け。

[実行例]

足し算を行います。

足す数を入力してください: 3

足す数を入力してください: 7

足す数を入力してください: -2

足す数を入力してください: 4

足す数を入力してください: 0

結果は12です。

参考課題3解答例

```
#include<stdio.h>
int main(void)
{
    int x;
    int ans;
    ans = 0;
    printf("足し算を行います。¥n");
    printf("足す数を入力してください: ");
    scanf("%d", &x);
    while(x != 0) {
        ans = ans + x;
        printf("足す数を入力してください: ");
        scanf("%d" , &x);
    }
    printf("結果は%dです。¥n" , ans);
    return 0;
}
```

[参考]

この講義では説明しませんが、do-while文を用いると、同じ内容のprintf文("足す数を入力してください: "の表示)を2か所に書かなくてよくなります。

参考課題4

キーボードから正の整数を受け取り、その数を合計値が100以上になるまで足し続け、合計値および何回足したかを表示するプログラムを書け。

[実行例]

正の整数を入力してください: 6

6を17回加えると102になります。

参考課題4解答例

```
#include<stdio.h>
int main(void)
{
    int x;
    int ans;
    int count;
    x = 0;
    ans = 0;
    count=0;
    printf("正の整数を入力してください: ");
    scanf("%d" , &x);
    while(ans < 100) {
        ans = ans + x;
        count = count + 1;
    }
    printf("%dを%d回加えると%dになります。¥n", x, count, ans);
    return 0;
}
```